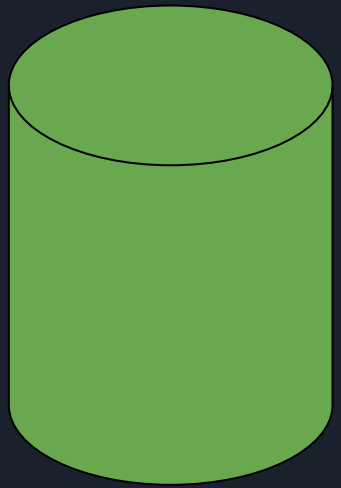
GESTIÓN DE DATOS

TRANSACCIONES

RECORDERIS...



¿QUÉ APRENDEREMOS HOY?



- Concepto: ACID - BASE
- Concepto: Transacción
- Concepto: Savepoint
- Sistemas monousuario vs multiusuario
- Palabras clave en mysql/mariadb para manejo de transacciones

REFLEXIÓN

Transferencia bancaria de $100 desde cuenta 1 hacia cuenta 2

```
update cuentas set saldo = saldo - 100 where id = 1;
-- falla en sistema

update cuentas set saldo = saldo + 100 where id = 2;
-- no hay fallas
```

1. ¿Se perdió dinero?
2. ¿De quién es la "culpa"?
3. ¿Se violó algún principio?
4. ¿Error de diseño?

CONCEPTO: Transacción



CONCEPTO: A.C.I.D.



CONCEPTO: B.A.S.E.



CONCEPTO: Transacción

Es una secuencia de operaciones que:

- **Se ejecuta completamente**
- **O no se ejecuta**

No existe el "más o menos" o "se ejecutaron 3 de 5 operaciones" o "quedó a medias"

CONCEPTO: Transacción

Ejemplos:

- **Un correo se envía o no se envía. No se envía "un pedazo"**
- **Una venta se completa o se rechaza. No es "media venta"**
- **Un estudiante queda matriculado en un curso o no queda. No queda "medio matriculado"**

CONCEPTO: A.C.I.D.

4 principios de bases de datos (generalmente relacionales):

Atomicity: Evitar operaciones "a medias". Es todo o nada.
Consistency: Asegurar que los datos guardados sean válidos
Isolation: Cada operación es independiente de la otra
Durability: Los datos escritos permanecerán en el tiempo

ACID no garantiza velocidad sino confiabilidad

CONCEPTO: B.A.S.E.

4 principios de bases de datos (generalmente no relacionales):

BAsically available: Garantiza disponibilidad del DBMS y datos
Soft state: Los datos pueden cambiar de forma *asíncrona*
Eventually consistent: Los cambios no se ven reflejados de inmediato en un sistema *distribuído*

BASE no garantiza coherencia sino disponibilidad

A.C.I.D. vs B.A.S.E.

	ACID	BASE
Modelo de datos de preferencia	Relacional	No relacional
Consistencia	Muy alta (fuerte)	Eventual (débil)
Disponibilidad	Más restrictiva	Más alta
Escalabilidad	Vertical (limitada)	Horizontal (flexible)
Transacciones	Más rigurosas	Más flexibles

Recordar: En bases de datos, "*débil*" **NO ES** sinónimo de "*malo*"

A.C.I.D. vs B.A.S.E.

¿Cual escoger?

(discusión verbal)

CONCEPTO: Transacción

Transacciones

- **Atomicidad:** N operaciones se ven como una sola
- **Consistencia:** Se evitan operaciones a "medio camino"
- **Concurrencia:** Un campo o registro se modifica por un usuario a la vez
- Se pueden **confirmar** o se pueden **deshacer**

CONCEPTO: Transacción

Multiusuario VS Monousuario



CONCEPTO: Transacción

- **Monousuario**
  - Diseñado para un usuario al tiempo
  - Los recursos son para ese único usuario
- **Multiusuario**
  - Diseñado para N usuarios simultáneos
  - Recursos compartidos entre los N usuarios

EJEMPLOS

- Monousuario
  - Aplicación de alarmas y recordatorios en dispositivo móvil
  - Hoja de cálculo para finanzas personales
- Multiusuario
  - Archivo compartido en google drive
  - Sistema de ventas en línea
  - Base de datos de biblioteca universitaria

PREGUNTA!!!

¿Un carrito de compras es monousuario o multiusuario?

PREGUNTA!!!

¿Una tienda en línea es monousuario o multiusuario?

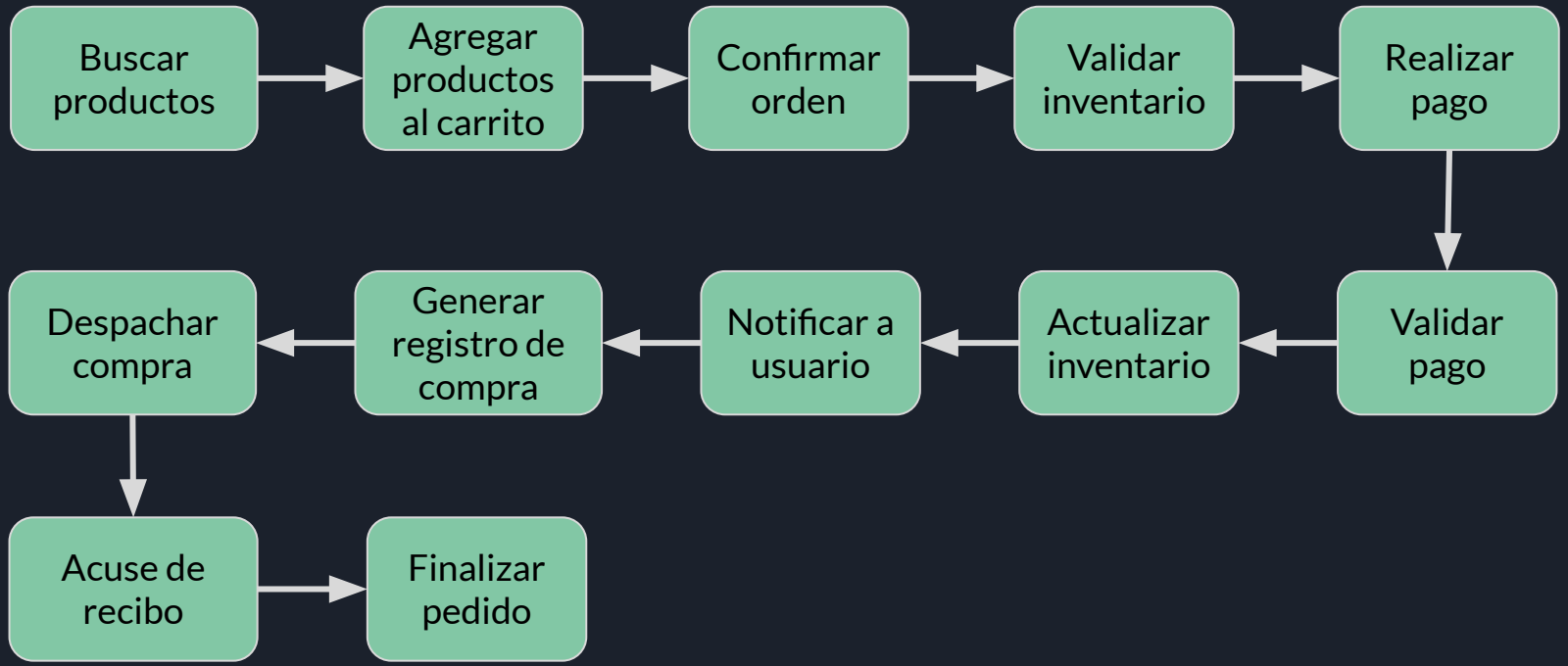
DEFINICIONES

read_item	Leer un objeto, variable o valor desde algún lado (archivo, base de datos, api, etc), y llevarlo a memoria
write_item	Escribir un valor de memoria en un medio de almacenamiento
read_set	Conjunto o grupo de read_item
write_set	Conjunto o grupo de write_item

EJERCICIO: Tienda en línea

Supongamos un escenario en el cual un usuario realiza una compra en línea de productos electrónicos. ¿Cuál sería el paso a paso para una transacción completa?

EJERCICIO: Tienda en línea



EJERCICIO: Tienda en línea

En el ejercicio anterior, ¿donde iniciaría una transacción y donde terminaría?

(Discusión verbal)

Transacciones: commit/rollback

start transaction	Punto de ***inicio*** de una transacción
commit	Finalizar la transacción, ***confirmando*** los cambios
rollback	Finalizar la transacción, ***deshaciendo*** los cambios

Transacciones: auto_commit

auto_commit

Variable de entorno que indica si una operación de modificación de datos (insert, update, delete) se confirma automáticamente o no.

NOTA: En mariadb/mysql están habilitada por defecto. Para deshabilitarlas debe hacerse manualmente

Transacciones en mariadb/mysql

IMPORTANTE!!!

Repite conmigo:

"Las transacciones aplican para las operaciones CRUD (datos), no para las operaciones DDL (estructura)"

Transacciones - ejemplo

Después de ejecutar el siguiente bloque de código, ¿con cuantos campos queda la tabla "people"?

```
create table people (
    id int not null,
    full_name varchar(50),
    primary key(id)
);
start transaction;
alter table people add column birth_date date not null;
rollback;
```

Transacciones - ejemplo

Después de ejecutar el siguiente bloque de código, ¿cuantos registros quedan en la tabla "people"?

```
insert into people (id, full_name) values (111, 'SHAKIRA');
start transaction;
insert into people (id, full_name) values (123, 'SELENA GOMEZ');
insert into people (id, full_name) values (456, 'PET SHOP BOYS');
rollback;
```

Transacciones - ejemplo

¿Qué sucede luego de ejecutar el siguiente bloque de código?

NOTA: Tomado de https://dev.mysql.com/

```
start transaction;

select  @a := sum(salary) from table1 where type = 1;
update table2 set summary = @a where type = 1;

commit;
```

Transacciones: Savepoint

Permite dividir una transacción en sub-transacciones

Transacciones - ejemplo savepoint

¿Qué sucede luego de ejecutar el siguiente bloque de código?

```
start transaction;

#Subtransacción 1: cantantes de pop
insert into singers1 (full_name, genre) values ('SELENA GOMEZ', 'POP'), ('ARIANA GRANDE', 'POP');
savepoint cantantes_pop;

#Subtransacción 2: cantantes de hip hop
insert into singers1 (full_name, genre) values ('CARDI B', 'HIP HOP'), ('POST MALONE', 'HIP HOP');
savepoint cantantes_hip_hop;

#Subtransacción 3: cantantes de reggaeton
insert into singers1 (full_name, genre) values ('BECKY G', 'REGGAETON'), ('KAROL G', 'REGGAETON');
savepoint cantantes_reggaeton;

rollback to cantantes_pop;

commit;
```

Transacciones: Tener en cuenta

"Una transacción debe durar lo menos posible"

¿Porqué?

- Evitar bloqueos innecesarios
- Mejorar el rendimiento en sistemas multiusuario

EJERCICIO

Dibujar un diagrama que explique cómo sería un proceso de transferir dinero de una cuenta a otra.

Posteriormente, identificar entre cuáles de esos pasos usted considera que debe haber una transacción (o más de una)

EJERCICIO (1/4)

Iniciar 2 sesiones en la herramienta de su preferencia (heidi, consola, etc)

En la sesión 1:

- Modificar la población de cualquier país ***sin usar transacciones***

En la sesión 2:

- Consultar la población de ese país después de ejecutar el update en la sesión 1

👉 Resultado esperado: La sesión 2 debe ver los cambios inmediatamente

EJERCICIO (2/4)

En la sesión 1:

- Modificar la población de otro cualquier país ***usando transacciones***. No ejecutar commit ni rollback por el momento

En la sesión 2:

- Consultar la población de ese país después de ejecutar el update en la sesión 1

👉 Resultado esperado: La sesión 2 debe ver la población anterior

- Volver a la sesión 1 y ejecutar commit. Repetir la consulta en la sesión 2.

👉 Ya los cambios si se deben ver reflejados en la sesión 2

EJERCICIO (3 / 4)

En la sesión 1:

- Modificar la población de otro cualquier país ***usando transacciones***. No ejecutar commit ni rollback por el momento

En la sesión 2:

- Consultar la población de ese país después de ejecutar el update en la sesión 1

👉 Resultado esperado: La sesión 2 debe ver la población anterior

- Volver a la sesión 1 y ejecutar rollback. Repetir la consulta en la sesión 2.

👉 No se debería ver reflejado ningún cambio en la sesión 2

EJERCICIO (4/4)

En la sesión 1:

- Aumentar en un 10% la población de COL, ARG y VEN. Usar savepoints entre cada operación. No ejecutar rollback ni commit por el momento

En la sesión 2: Consultar la población de los 3 países en una sola sentencia sql
👉 La sesión 2 debe ver la población anterior de cada país (sin cambios aún)

En la sesión 1: ejecutar el rollback hacia cualquier savepoint.
👉 La sesión 2 aún no debe ver cambios

En la sesión 1: ejecutar commit.
👉 La sesión 2 debe ver los cambios hasta el savepoint seleccionado

¿PREGUNTAS?

